Title: Sentiment Analysis using PySpark

This project was developed to explore the research question:

How accurately can we classify the sentiment of text using machine learning models in a distributed PySpark environment?

Millions of people posted their opinion in the form of tweets and textual arguments online, identifying whether a text phrase is positive or negative can be valuable for businesses, researchrs, etc. The purpose of my project is to build a fast and scalable sentiment classification system using PySpark, which is designed to handle big data efficiently.

I began by applying traditional machine learning models such as Logistic Regression, SVM, Naive Bayes, and Random Forest on a dataset containing 1.6 million labeled tweets. To speed up training and make full use of CPU cores, I integrated multithreading using python's concurrent tools. This approach helped me to train several models simultaneously, improving the overall processing time and resource usage. Initially, I had the motivation to implement a neural network-based models using pytorch. But pyspark does not support deep learning natively, and converting spark dataframes to pandas for pytorch input introduces compatibility issues. This was a key limitation I faced, which stopped me from integrating neural networks into the current pipeline. After this initial training, I observed that models like SVM, Logistic Regression, and Naive Bayes gave relatively better performance but decision tree underperformed might be due to overfitting. Each model evaluated using accuracy, F1-score, and the confusion matrix.

To improve model performance, I used hyperparameter tuning with cross-validation. The impact was clear, some models like logistic regression improved noticeably after tuning, becoming more reliable. Others showed little improvement but decision trees reduced performance highlighting that tuning is not always beneficial. Overall, the tuning approach performed efficiently and gave more balanced results compared to the initial training.

By developing this project, I gained experience in handling real world textual data, learned to design parallel pipelines for model training, and learned about the ML models and tools like PySpark, multithreading etc. In future, this project could be enhanced by integrating deep learning frameworks that are better suited for text data, like LSTM and transformer models and also extend this work into multilingual sentiment analysis or beyond binary classification to multi class classification.

References

Pang, B., & Lee, L. (2008). *Opinion mining and sentiment analysis*. Foundations and Trends in Information Retrieval, 2(1–2), 1–135. https://doi.org/10.1561/1500000011

Zhang, L., Wang, S., & Liu, B. (2018). *Deep learning for sentiment analysis: A survey*. Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery, 8(4), e1253. https://doi.org/10.1002/widm.1253

Configuration - Spark 4.0.0 Documentation

7.3. Preprocessing data — scikit-learn 1.7.0 documentation

Tokenizer — PySpark 4.0.0 documentation

HashingTF — PySpark 4.0.0 documentation

IDF — PySpark 4.0.0 documentation

MLlib (DataFrame-based) — PySpark 4.0.0 documentation

Pipeline — PySpark 4.0.0 documentation

concurrent.futures — Launching parallel tasks — Python 3.13.5 documentation

CrossValidator — PySpark 4.0.0 documentation

ParamGridBuilder — PySpark 4.0.0 documentation

MulticlassClassificationEvaluator — PySpark 4.0.0 documentation

Hyperparameter Tuning - GeeksforGeeks